

Endangered Species Prediction Pipeline: Leveraging Edge Detection, CNNs, and Neural Networks

Avanish Grampurohit

EE456

12/17/2024

Abstract

This project aims to develop a robust neural network pipeline for classifying species from images in the Trap image dataset and leveraging these classifications to predict whether the species is endangered. The first phase involves training a convolutional neural network (CNN) to categorize species based on image features. In the second phase, the model's output probabilities are reverse-engineered to predict the endangered status of each species, providing valuable support for conservation efforts. This project explores the potential to automatically flag images that may feature endangered species, thereby helping identify areas of concern and alerting officials to facilitate preservation. The pipeline developed in this project is the result of an experimental approach, testing multiple strategies to enhance efficiency. The outcomes demonstrate a significant reduction in training time (by approximately 66%) and achieve performance levels comparable to state-of-the-art algorithms. This work successfully addresses the challenge of predicting endangered species from images, a typically uncharted area of research, using reverse engineering techniques.

Contents

1	Outline	1
2	Model Architecture and Experiments	2
2.1	Dataset	2
2.2	Architecture	3
2.3	Data Processing	3
2.4	Edge Detection	3
2.5	CNN Model	4
2.6	Feed-Forward Network	4
2.7	Experiments	5
3	Results	5
3.1	Edge Detection Results	5
3.2	CNN Model	5
3.3	Feed-Forward Network Results	6
3.4	Final Results	6
4	Conclusions	7

1 Outline

This project focuses on predicting endangered species using Trap images, which are photographs captured by camera traps. Camera traps are automated remote cameras triggered by movement or heat, frequently deployed for ecological monitoring and wildlife studies. However, these images are often of lower quality and captured at random angles, making accurate classification a challenging task. The aim of this project is to develop a robust and performance-optimized system capable of identifying endangered species from such images without compromising effectiveness.

The methodology begins with leveraging edge detection techniques and preprocessing the images by converting them to resized, black-and-white formats. This step ensures that the image features are simplified while retaining critical details for classification. A con-

volutional neural network (CNN) is first employed to predict the species and generate the top five class probabilities. These predictions, along with their respective confidence scores and categorical labels such as mammals, snakes, and others, are then used as input into a simple feed-forward neural network. This second-stage model effectively determines whether the input image corresponds to an endangered species based on the predicted species probabilities and categorical information.

To implement this system, the project utilizes the Ohio Small Animals Dataset (lila.science), a publicly available dataset comprising 118,554 images captured by AHDriFT camera traps deployed across Ohio. This dataset includes 45 species spanning 7 categories, offering a rich and diverse collection of data for training and evaluation.

The project adopts a well-rounded computational workflow, implemented using Python as the programming language. Tools such as

OpenCV are employed for image preprocessing and edge detection, while PyTorch is used to construct and train neural network models. For seamless experimentation and resource management, Jupyter Notebook and cloud-based platforms like Google Colab and Google Drive are utilized. To further improve computational efficiency, concurrent programming with ProcessPool is employed. Additionally, statistical libraries such as NumPy, Pandas, and Matplotlib are used for data manipulation, analysis, and visualization. Finally, scikit-learn is leveraged to compare the performance of the proposed system against other widely used machine learning techniques.

This work presents a novel approach to endangered species identification by combining image classification and predictive modeling in a streamlined pipeline. The proposed system demonstrates the potential to automate species classification while reliably identifying endangered species, contributing to wildlife conservation efforts.

2 Model Architecture and Experiments

This section describes the thought process behind the experiments and implementation of the final design.

2.1 Dataset

The dataset utilized in this study consists of 118,554 images captured by the AHDriFT camera traps deployed across various locations in Ohio. The AHDriFT system employs a fence that directs small animals into an enclosure, where a downward-facing camera captures images of the animals.

Each image is labeled with one of 45 species, including notable labels such as the Eastern garter snake (31,899 images), song sparrow (14,567 images), meadow vole (14,169 images), and a blank label for non-animal images (11,448 images). The dataset features a wide variety of species and environmental con-

ditions, making it well-suited for tasks such as species identification and ecological monitoring.

The images are collected from 168 unique camera locations, each identified by a distinct location ID. A deployment typically consists of two or three cameras arranged in a linear or Y-array configuration. For example, the location IDs “KPC1”, “KPC2”, and “KPC3” correspond to separate cameras within the same deployment. The diversity of camera locations and configurations provides additional context for model training and evaluation, enabling the analysis of spatial patterns and the effect of different camera placements on image capture.^[1]

2.2 Architecture

The architecture of the entire pipeline is illustrated in Figure 1. The process begins with data extraction and preprocessing, followed by edge detection to convert all images to grayscale and highlight features as edges. The images are then passed through a basic convolutional neural network (CNN), which generates the top 5 confidence scores for each prediction. Finally, the data is organized to facilitate the prediction of images depicting endangered species.

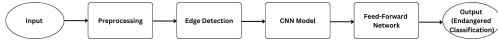


Figure 1: Model Pipeline

2.3 Data Processing

The dataset consists of images representing one of 45 distinct species. These images are organized within a structured file path, allowing for the extraction of category information. In addition, a metadata JSON file is utilized to generate two essential CSV files. The first, labels.csv, contains information on the species name, file path, and image file details. The second, category_annot.csv, includes the species names along with their corresponding category IDs.

To ensure uniformity and improve computational efficiency, the images are read using OpenCV and resized to a resolution of 128x128. Each image is then converted to grayscale before undergoing edge detection, which emphasizes key visual features for subsequent analysis.

Following the execution of the CNN model, its output is combined with the labels.csv, category_annot.csv, and the confidence scores generated during the model's predictions to create a merged dataset. This process involves reverse-engineering the category information by associating each image with its predicted

category. The resulting dataset integrates image details, category information, and confidence scores as inputs, while the endangered status serves as the output. The final dataset, referred to as merged_df3, is depicted in Figure 2 and forms the primary data source for the final computation.

	predicted labels	confidence scores	labels	endangered	category
0	[31, 11, 20, 12, 36]	[0.93757445, 0.02148269, 0.015433677, 0.005771...	31	0	serpentes
1	[31, 39, 42, 40, 33]	[0.9843252, 0.014914763, 0.00025533908, 0.0002...	31	0	serpentes
2	[11, 31, 12, 26, 23]	[0.99921894, 0.00053616933, 5.301732e-05, 4.85...	11	0	invertebrate
3	[20, 18, 26, 11, 28]	[0.99965274, 0.00015542262, 0.00013579919, 3.3...	20	1	mammalia
4	[26, 28, 21, 18, 8]	[0.9812626, 0.0098423455, 0.0041365544, 0.0028...	26	0	mammalia
...	...	...	...	...	...
214215	[31, 18, 27, 11, 29]	[0.99998593, 1.3415117e-05, 1.9236344e-07, 1.6...	31	0	serpentes
214216	[25, 17, 21, 13, 22]	[0.999509, 0.00032877346, 6.7975445e-05, 5.535...	25	1	mammalia
214217	[31, 40, 35, 39, 38]	[0.9999907, 7.11941e-06, 2.119325e-06, 2.00449...	31	0	serpentes
214218	[26, 9, 18, 20, 42]	[0.9999546, 4.411196e-05, 7.528964e-07, 2.0205...	26	0	mammalia
214219	[31, 40, 42, 39, 26]	[0.9999366, 6.279316e-05, 5.8436177e-07, 2.123...	31	0	serpentes

Figure 2: merged_df3

2.4 Edge Detection

Edge detection is a fundamental technique in computer vision used to identify the boundaries or edges within an image. This process is typically achieved by applying a Gaussian filter to the image, followed by detecting significant changes in intensity. Intensity variations at a given scale can be effectively captured by identifying the zero-crossings of the Laplacian operator, $\Delta G(x, y) * I(x, y)$, where $G(x, y)$ represents a two-dimensional Gaussian distribution, and Δ^2 denotes the Laplacian operator^[3] (Laplacian method). For this study, the Canny edge detection method, implemented through OpenCV^[4], will be employed. The Canny method uses the following formulas for gradient-based edge detection:

$$M = \sqrt{G_x^2 + G_y^2}$$

$$\theta = \arctan\left(\frac{G_y}{G_x}\right)$$

Edge detection is applied to all images within the dataset to extract key outlines and features of each species. This preprocessing step eliminates color information, converting the

images to grayscale, which reduces the number of channels from three to one. As a result, the dataset size is significantly reduced from 3.3GB to 783MB, facilitating more efficient storage and processing.

2.5 CNN Model

The Convolutional Neural Network (CNN) model used for this image classification task consists of three convolutional layers followed by two fully connected layers. The model starts by applying three convolutional layers with increasing filter sizes (32, 64, and 128 filters) to extract spatial features from the input grayscale images. Max-pooling layers are used after each convolutional layer to down-sample the spatial dimensions. The output is then flattened and passed through two fully connected layers, with the final layer producing class predictions. The model is trained using the Adam optimizer and cross-entropy loss, and the training loop evaluates accuracy on both the training and validation datasets. The architecture for this model is shown in Figure 3. This dataset is inspired by the Convolutional neural network (cnn) for image detection and recognition paper^[2].

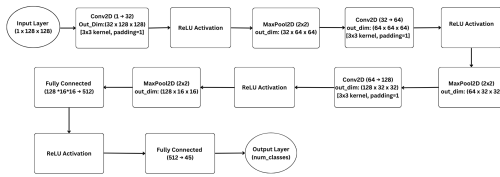


Figure 3: CNN Architecture

The dataset is split into training and validation sets using an 80-20 split, with data being normalized to the range $[0, 1]$. The images are processed using a custom ‘ImageDataset’ class and loaded into PyTorch’s ‘DataLoader’ for efficient batching. The training loop runs for 10 epochs, where the model’s performance

is tracked through training and validation accuracy. The top 5 predicted labels and their confidence scores are stored during the validation phase to provide additional insights into the model’s predictions as shown in Figure 4. These top 5 predictions help evaluate the model’s confidence in its classifications.

```
Image 1:
True Label: 31
Predicted Labels: [31 11 20 12 36]
Confidence Scores: [0.93757445 0.02148269 0.01543368 0.00577199 0.00458528]

Image 2:
True Label: 31
Predicted Labels: [31 39 42 40 33]
Confidence Scores: [9.8432517e-01 1.4914763e-02 2.5533908e-04 2.0105907e-04 1.8435830e-04]

Image 3:
True Label: 11
Predicted Labels: [11 31 12 26 23]
Confidence Scores: [9.9921894e-01 5.3616933e-03 5.3017320e-05 4.8510399e-05 4.6607616e-05]

Image 4:
True Label: 20
Predicted Labels: [20 18 26 11 28]
Confidence Scores: [9.9965274e-01 1.5542262e-04 1.3579919e-04 3.3500393e-05 1.5478588e-05]

Image 5:
True Label: 26
Predicted Labels: [26 28 21 18 8]
Confidence Scores: [9.8126262e-01 9.8423455e-03 4.1365544e-03 2.8688153e-03 6.7288073e-04]
```

Figure 4: Confidence Score examples of Images

The model’s performance is visualized by plotting the training and validation accuracies over the course of the epochs.

2.6 Feed-Forward Network

The dataset is divided into training and validation sets using an 80-20 split, with 80% allocated to the training set and 20% to the validation set. For both sets, PyTorch datasets and data loaders are created using the ‘AnimalDataset’ class, with a batch size of 64. The neural network model, defined by ‘NNModel’, takes as input the number of features in the dataset and outputs the number of unique classes. The model is trained with the Adam optimizer at a learning rate of 0.001 and cross-entropy loss as the loss function. The architecture for this model is shown in Figure 5.

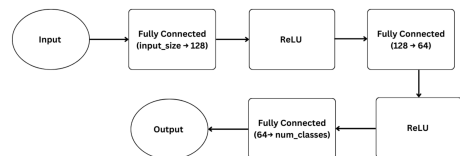


Figure 5: Feed-Forward Architecture

The model's performance is visualized by plotting the training and validation accuracies over the course of the epochs

The training loop runs for 1000 epochs, although experiments with fewer epochs (100 and 10) revealed no significant difference in performance. The model's training accuracy is calculated by comparing predicted labels with the ground truth, and validation accuracy is computed similarly on the validation set. Loss and accuracy metrics for both training and validation are printed every 100 epochs to monitor performance. The model is trained on either a GPU or CPU, depending on the availability of CUDA.

To visualize the model's learning progress and generalization ability, accuracy curves for both training and validation sets are plotted over the course of the training process.

2.7 Experiments

Two experiments were conducted: one focused on evaluating the performance of the model, and the other compared its results with those of other popular machine learning techniques. The first experiment explored different image formats, including colored (RGB), grayscale (black and white), and grayscale with edge detection applied. The file sizes of the resulting datasets were compared, and datasets with file sizes under 1GB were selected for accuracy testing. In the second experiment, the model's final accuracy in predicting images of endangered species was compared against other machine learning models, including Support Vector Machine (SVM), Random Forest, and Logistic Regression.

3 Results

This section describes the results of experiments and performance of models.

3.1 Edge Detection Results

Edge detection was performed on the images using `cv2.Canny()` after converting them to grayscale. This process identifies areas of high intensity change, highlighting the edges while suppressing the rest of the image, effectively converting it to a black-and-white format. The edge detection, combined with additional image preprocessing, significantly reduced the size of the dataset. The original 7.8 GB zip file was compressed to a 124 MB file containing the edge-detected image matrices. This reduction in file size improved overall processing efficiency, reducing classification time by approximately 66%.

92.22% as shown in figure 6. This is comparable to the benchmark of 99.6%^[2] trained over 50 epochs vs our model over 10 epochs. The experiments show that the colored and grayscale images take too long to train and hence have not been tested for accuracy scores as they lack the performance the model is meant to achieve.

3.2 CNN Model

CNN model performs extremely well with the edge detected images having a training accuracy of 99.44% and Validation accuracy of

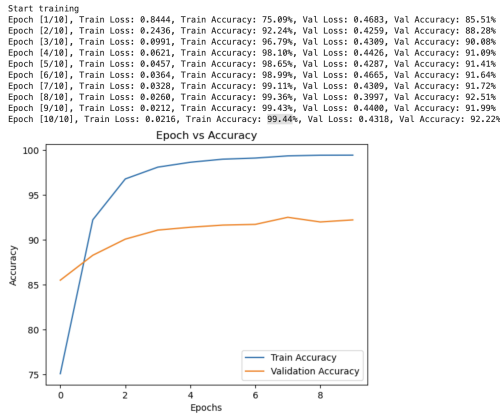


Figure 6: CNN: Training and Validation accuracies for each epoch

3.3 Feed-Forward Network Results

The feed-forward neural network demonstrates strong performance and accuracy; however, it may not be the optimal choice for this step of the pipeline. While its accuracy is comparable to that of a Random Forest classifier, the training and validation accuracies exhibit minimal improvement over epochs and are notably noisy. This behavior is attributed to the simplicity of the model, where, in most cases, a single parameter (the high-confidence label with a score exceeding 0.9) dominates the prediction. Since this parameter often corresponds to the true label, the task of predicting whether an image contains an endangered species reduces to a simple mapping problem.

Nevertheless, the feed-forward network achieves a validation accuracy of 97.44%, which surpasses the 92.22% accuracy obtained by relying solely on the CNN and then performing direct mapping. This result highlights the effectiveness of incorporating this approach into the pipeline, significantly enhancing the model's ability to detect images depicting endangered species. The training and validation accuracies are visualized in Figure 7, demonstrating the impact of this method on the overall classification process.

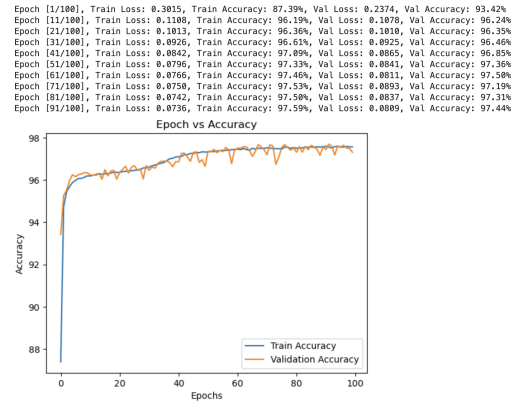


Figure 7: Feed-Forward: Training and Validation accuracies for each epoch

3.4 Final Results

The final results reveal that, while the feed-forward neural network offers a promising approach, a Random Forest classifier may be a more suitable choice for this stage of the pipeline. Random Forest models are known for their robustness, stability, and ability to handle feature importance in a more interpretable manner. The experimental results indicate that the Random Forest classifier achieves higher stability in training and validation accuracy, with less fluctuation observed in the accuracy plot compared to the feed-forward network. Figure 8 shows the comparison of accuracies achieved by different models.

Model	Accuracy
Feed-Forward NN	97.44%
Random Forest	97.86%
SVM	84.46%
Logistic Regression	83.19%

Figure 8: Accuracy scores for different models

It is important to note that the final model employs a feed-forward neural network for this step, as the purpose of this study is to explore experimental approaches and compare a pure neural network-based approach with alternative mixed-method approaches. The goal is

not solely to construct the "ideal model" but rather to investigate and evaluate different techniques for solving the problem. This experimental framework allows for a more

comprehensive understanding of model performance and informs future improvements to the overall pipeline.

4 Conclusions

This study aimed to evaluate the performance of a combined neural network architecture with edge detection for predicting the presence of endangered species in images. By leveraging edge-detected inputs and structured features like category and confidence scores, the model achieved higher accuracy in endangered species detection compared to direct classification-to-mapping approaches. While the feed-forward network showed some instability in training, the model still outperformed baseline methods, demonstrating the potential of this hybrid approach.

The results highlight the effectiveness of combining neural networks with auxiliary features, though Random Forest models showed better performance in some cases. Despite this, the study provides valuable insights into how machine learning can support wildlife conservation. Deploying such models on trap camera datasets could enable automated detection of endangered species in forested areas, supporting timely intervention and improved conservation strategies. Future work will explore larger datasets, more species classes, and optimized model architectures to further enhance performance a threshold value for confidence scores can also be used to only use the low confidence images for feed-forward to improve the performance even further.

References

- [1] S. Alasubramaniam. Optimized classification in camera trap images: An approach with smart camera traps, machine learning, and human inference. Master's thesis, The Ohio State University, 2024.
- [2] Rahul Chauhan, Kamal Kumar Ghanshala, and R.C Joshi. Convolutional neural network (cnn) for image detection and recognition. *IEEE*, pages 278–282, Dec 2018.
- [3] David Marr and Ellen Hildreth. Theory of edge detection. *Proceedings of the Royal Society of London. Series B. Biological Sciences*, 207(1167):187–217, 1980.
- [4] openCV. opencv library for edge detection using the canny method, 2024. Accessed: 2024-12-17.